Comparing Approaches for Kubernetes Cluster Fleet Management

26 November 2024 - ID G00820793 - 37 min read

By: Tony Iams

Initiatives: Technologies and Markets

As Kubernetes adoption grows, I&O technical professionals increasingly find they must deploy and operate multiple Kubernetes clusters. This research reviews possible approaches for managing Kubernetes fleets and provides guidelines for matching the most suitable approach with a particular use case.

Overview

Key Findings

- I&O technical professionals must operate multiple Kubernetes clusters to address a number of use cases, including multitenancy and disaster recovery (DR), across hybrid, multicloud or edge scenarios. Each use case brings a different set of challenges and a choice of approaches for implementing solutions.
- A complete Kubernetes fleet management solution must provide several technical capabilities, including distributed life cycle and state management, multicluster networking and ingress, and related operations for centralized observability and storage and data management. However, not all capabilities will be required for every use case.
- Although a fleet-based solution for multitenancy offers security and reliability benefits, it introduces more administration complexity compared with shared clusters. This solution also assumes that tenant teams have the expertise to operate Kubernetes clusters themselves, which is most suitable in engineering-driven organizations, where knowledge about using Kubernetes is widespread.

This research note is restricted to the personal use of .

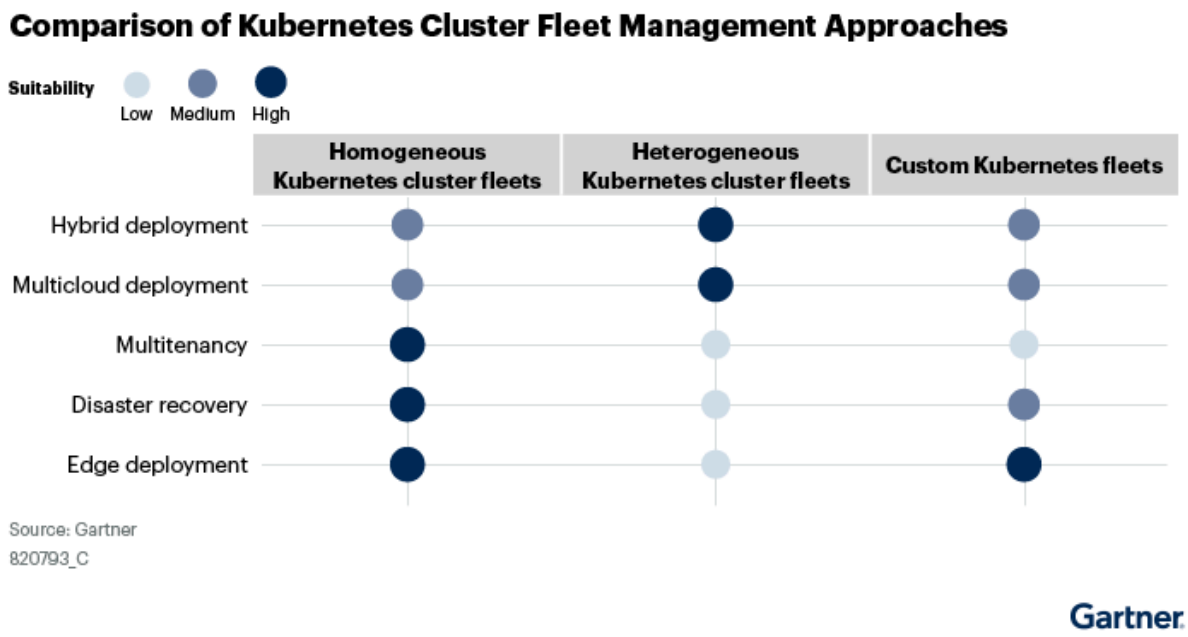
Recommendations

- Implement isolated multitenancy, DR and some edge computing use cases by deploying fleets of clusters running the same Kubernetes distribution.
- Target hybrid and multicloud fleet management use cases by selecting a solution that can manage fleets of clusters running different Kubernetes distributions.
- Assess the feasibility of using a custom approach for edge use cases by engineering a solution using open-source tools, but only if the organization has sufficient skills to support the solution internally.

Comparison

Gartner has identified three possible approaches for managing fleets of Kubernetes clusters (homogeneous Kubernetes cluster fleets, heterogeneous Kubernetes cluster fleets and custom Kubernetes fleets), and each approach varies in its suitability for different fleet management use cases (see Figure 1).

Figure 1: Comparison of Kubernetes Cluster Fleet Management Approaches



The three possible fleet management approaches are:

- **Homogeneous Kubernetes cluster fleets:** With this approach, operators deploy multiple clusters running the same distribution of Kubernetes, using a product or service that includes the necessary tools to manage fleets of clusters running that distribution.

This research note is restricted to the personal use of .

- **Heterogeneous Kubernetes cluster fleets:** With this approach, operators deploy a third-party product or service that is designed to manage multiple clusters running different distributions of Kubernetes.
- **Custom Kubernetes fleets:** With this approach, operators use open-source tools to arrive at a custom-made solution to manage multiple clusters for their specific use case.

Each of these approaches has trade-offs, in terms of meeting the requirements of use cases that Gartner has identified for Kubernetes fleet management.

Hybrid Deployment

With this use case, organizations seek the choice of running containers on Kubernetes clusters deployed on-premises or on a public cloud service. The hybrid use case is best supported by an approach based on managing heterogeneous Kubernetes cluster fleets. The major public cloud services all offer native-managed Kubernetes distributions that are deeply integrated with their respective platforms, while self-managed Kubernetes distributions deployed on-premises are optimized to run on data center or edge infrastructure.

To best handle the divergent capabilities of these environments, operators should select a management platform that can normalize the operational experience of different Kubernetes distributions. The solution should be able to operate clusters running heterogeneous distributions with a “single pane of glass,” while also providing the full set of capabilities needed to manage multiple clusters in a fleet.

The homogeneous Kubernetes cluster fleet approach may be applicable if the single distribution selected can be deployed the same way both on-premises and with public cloud infrastructure. However, this approach may incur compromises in terms of higher acquisition costs, limited depth of integration with on-premises infrastructure of the cloud service or limited support for delivery as a managed service in the cloud.

Multicloud Deployment

This research note is restricted to the personal use of .

With this use case, operators want to leverage the Kubernetes standard to provide a consistent operating environment for containerized applications across multiple cloud infrastructures from different vendors. Because each cloud service has unique points of integration with its managed Kubernetes services, the multicloud use case is best supported by an approach supporting heterogeneous Kubernetes cluster fleets. The homogeneous Kubernetes cluster fleet approach may be applicable if the single distribution selected can be deployed the same way on multiple cloud services, but the same caveats apply as described above for the hybrid use case.

Isolated Multitenancy

With this use case, operators seek to reduce the risk of catastrophic failure by assigning separate clusters to different teams or classes of applications. Because each tenant expects to have the same operational experience, the multitenancy use case is best supported by the homogeneous Kubernetes cluster fleet approach. When deploying the same distribution for each cluster in the fleet, policies and cluster extensions can be applied consistently for each tenant. The use of homogeneous distributions also simplifies the implementation of virtual clusters using technologies such as Loft (vCluster) or kcp, where a Kubernetes control plane is used to manage other Kubernetes distributions. [1]

Disaster Recovery

While the Kubernetes architecture supports high levels of uptime for applications running within a cluster, it does not extend to DR (i.e., providing resiliency against failures that affect entire sites or regions). It has not been practical to stretch the control plane of a single Kubernetes cluster across geographically significant distances. Therefore, to implement DR with Kubernetes, it is necessary to set up two or more clusters, and use specialized tools that are not part of Kubernetes to replicate the application state and data between clusters.

The Kubernetes architecture does not specify how backup or DR should be implemented with clusters. Therefore, I&O technical professionals will have to arrive at their own solutions, based on the needs of applications deployed on Kubernetes clusters. However, the homogeneous Kubernetes cluster fleet approach will be easiest to implement. Alternatively, a custom Kubernetes fleet management approach can be used, since the DR configuration will be limited to two or three clusters. The primary fleet management functions will be replication of application state and data between the primary and backup cluster.

Edge Deployment

This research note is restricted to the personal use of .

Containers and Kubernetes are well-suited for edge computing architecture because they provide a consistent environment in which applications run from development to production. When Kubernetes is deployed at the edge, some implementations may require hundreds or thousands of Kubernetes clusters to be managed remotely. In this scenario, either the homogeneous Kubernetes cluster fleet or custom Kubernetes fleet management approach will be the best solution. Given the many technical challenges in mirroring cloud infrastructure at the edge in order to run containers and Kubernetes, Gartner generally recommends using an integrated stack from a single vendor as much as possible to simplify the implementation. However, a custom Kubernetes fleet management approach may be necessary in order to meet the unique requirements of the containerized edge application.

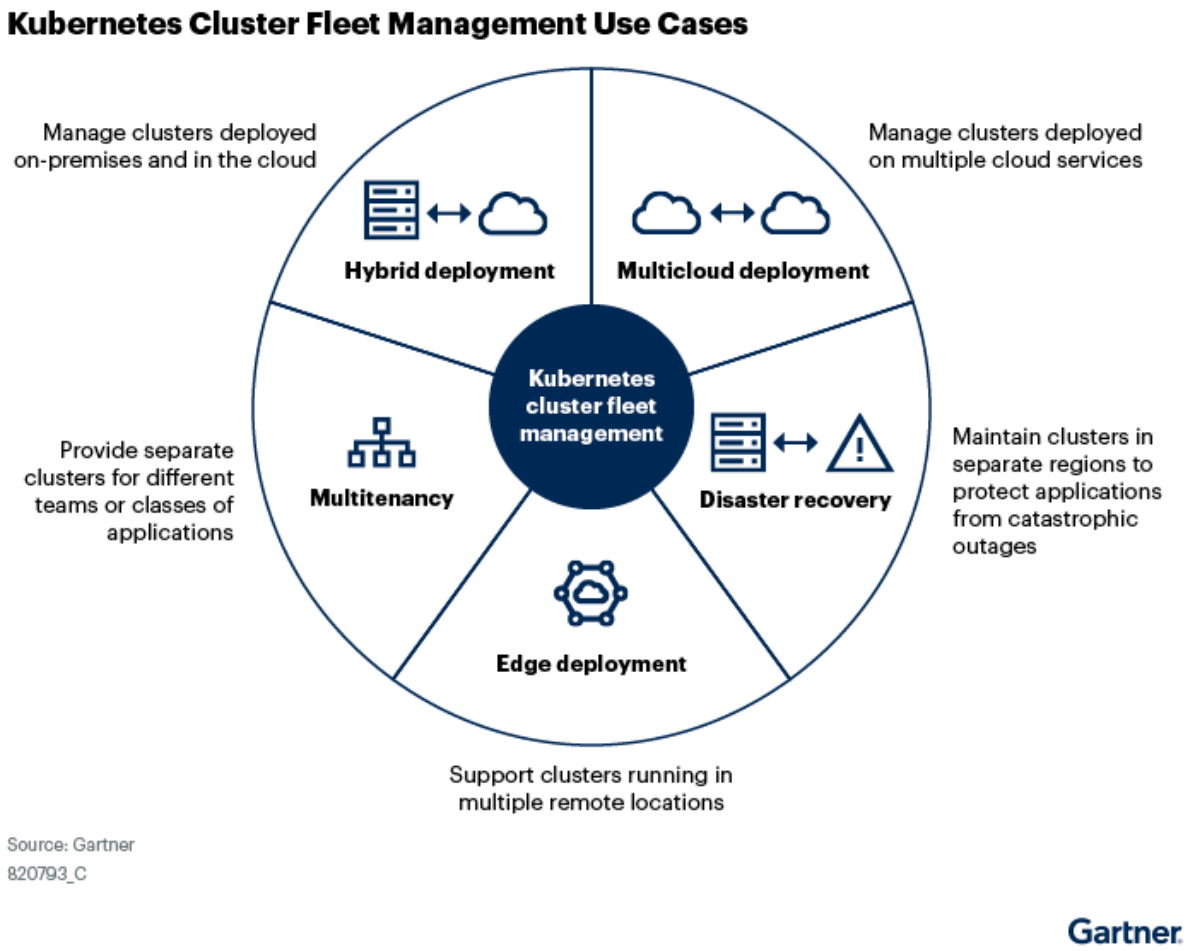
Analysis

As Kubernetes adoption grows, I&O technical professionals increasingly find they must deploy and operate multiple Kubernetes clusters potentially derived from different Kubernetes distributions and products, either in a single region on-premises or in the cloud, or across multiple regions. Managing Kubernetes cluster fleets will be required for several possible use cases (see Figure 2), including

- Hybrid deployment of clusters
- Multicloud deployment of clusters
- Using separate clusters to implement multitenancy
- DR between clusters in different locations
- Deploying clusters at the edge

This research note is restricted to the personal use of .

Figure 2: Kubernetes Cluster Fleet Management Use Cases



Use Cases for Kubernetes Fleets

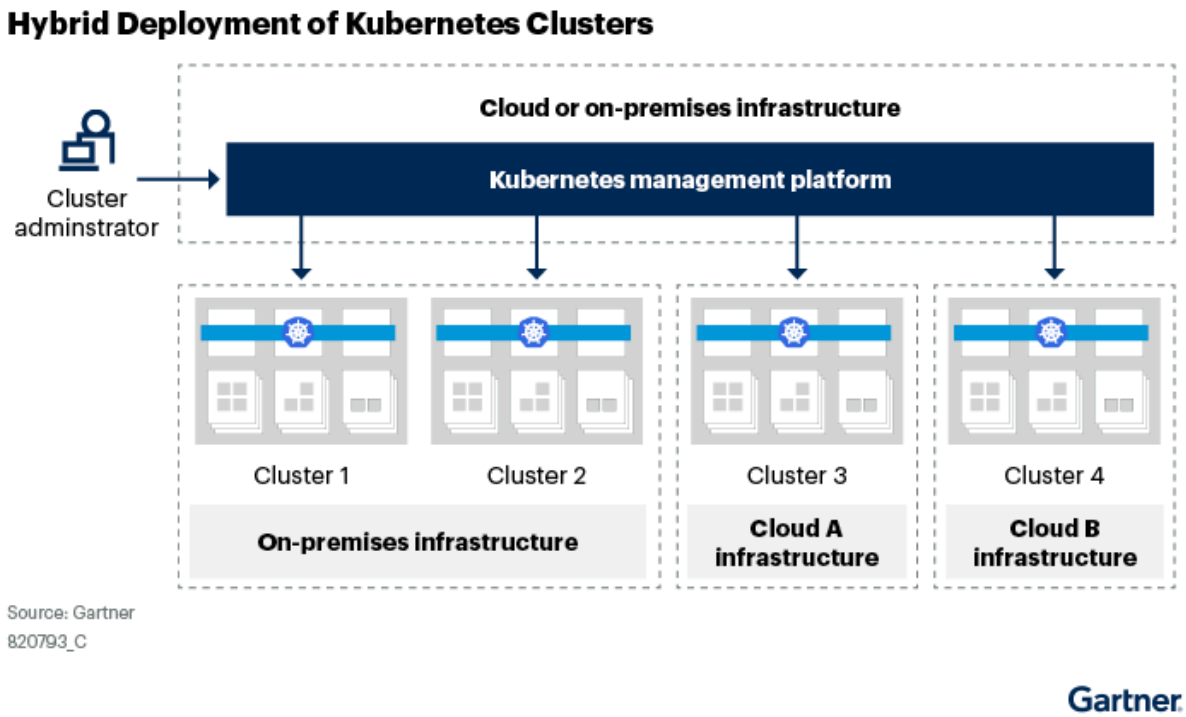
Each use case brings a different set of challenges, and a choice of approaches for implementing solutions.

Hybrid Deployment Use Case

One of the most common use cases for Kubernetes cluster fleet management is with hybrid deployments. Kubernetes runs equally well on-premises and on public cloud platforms, either as a managed service or self-managed on infrastructure as a service (IaaS). Organizations often begin their deployments of containers and Kubernetes either on-premises or in the cloud, and as their cloud-native initiatives scale, they start moving toward a hybrid approach. Splitting the control and data planes of a single Kubernetes cluster between on-premises infrastructure and the cloud is impractical in most cases (although some initiatives for this approach have emerged). Therefore, hybrid scenarios require separate clusters to be deployed and operated on-premises and in the cloud (see Figure 3).

This research note is restricted to the personal use of .

Figure 3: Hybrid Deployment of Kubernetes Clusters



For organizations running a self-managed Kubernetes distribution on-premises, the question then becomes whether they run that same distribution in the public cloud or use the native Kubernetes distribution offered on their cloud platform as a managed service. Many Gartner clients favor the latter approach because they expect that the native Kubernetes service will have the deepest possible integration with the cloud infrastructure, as well as other native services offered on that cloud. In addition, clients have found that the cost of acquisition is lower for the cloud's native Kubernetes service than other options, and that they can't always innovate at the speed of the cloud provider (e.g., support new virtual machine instance types).

This research note is restricted to the personal use of .

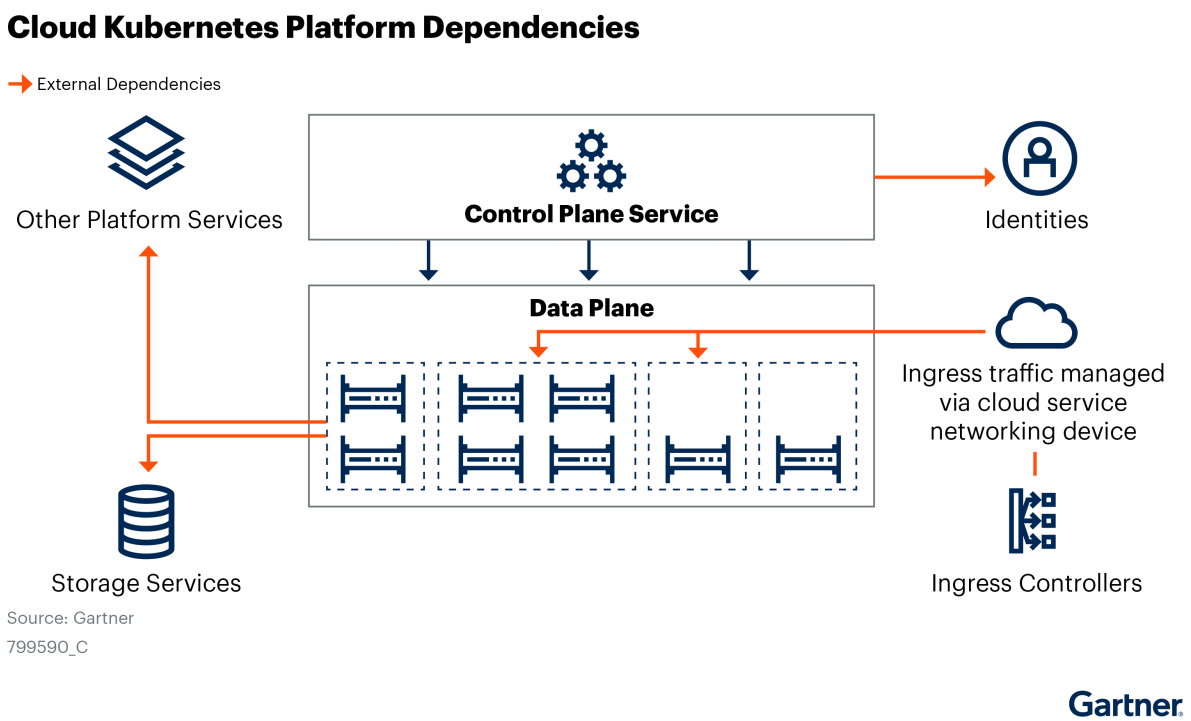
Organizations that start their Kubernetes initiatives with the native service on a public cloud, and then want to expand their deployments to on-premises infrastructure, face a choice. All the major cloud service providers now offer options for running their Kubernetes distribution on-premises. Examples include Amazon EKS Anywhere, Microsoft Azure Kubernetes Service (AKS) on AzureStack HCI and Google Kubernetes Engine (GKE) in Google Distributed Cloud. Some Gartner clients have evaluated running these cloud Kubernetes distributions on-premises in some form, but found that these options have not yet reached the maturity that they expected. As an alternative, organizations adopt a separate distribution for on-premises deployment that is optimized for data center or edge infrastructure. In this case, a fleet management solution that can normalize the operational experience across multiple distributions will provide the most effective choice.

Multicloud Deployment Use Case

With this use case, organizations seek to leverage the standard Kubernetes API to provide a consistent operating environment for containers across different public cloud infrastructures. With the ability to move a Kubernetes application from a cluster in one cloud service to another, users hope to reduce lock-in by minimizing switching costs. However, while Kubernetes provides standard APIs to abstract compute storage and networking infrastructure, any Kubernetes-based platform running in the cloud will have unique points of integration with that particular cloud service (see Figure 4).

This research note is restricted to the personal use of .

Figure 4: Cloud Kubernetes Platform Dependencies



With the multicloud deployment use case, the Kubernetes fleet management solution must include a way to configure these integration points on each cloud service in order to provide a consistent operational experience across clouds. These integration points include:

- **Identities:** Integration with the native identity mechanism for the cloud service, such as users and service accounts, will be needed to validate access to the Kubernetes API. Identity integration will be required for cluster life cycle management activities such as starting, stopping and upgrading clusters.
- **Ingress controllers:** All the major cloud service providers provide integration with their network fabric and load balancers to manage ingress traffic to Kubernetes clusters, and these points of integration should be manageable in a consistent way across cloud services.
- **Storage services:** To support stateful applications that need to persist data beyond the lifetime of a container, Kubernetes provides a standard method to access cloud storage services through persistent volume claims (PVCs) and container storage interface (CSI) drivers. While these abstractions are standardized, their configuration will be unique to each cloud service.

This research note is restricted to the personal use of .

- **Other cloud platform services:** A cloud Kubernetes service can integrate with other native platform services on the cloud, such as logging, monitoring, health checks, registries and secrets management.

The native Kubernetes services on each of the major cloud platforms have all been engineered with these points of integration documented and tested. Therefore, a fleet management solution that can normalize the operational experience across multiple cloud Kubernetes services will provide the most effective choice for addressing this use case. For further insight on portability of Kubernetes-based applications between cloud services, see Assessing Kubernetes for Hybrid and Multicloud Application Portability.

Isolated Multitenancy Use Case

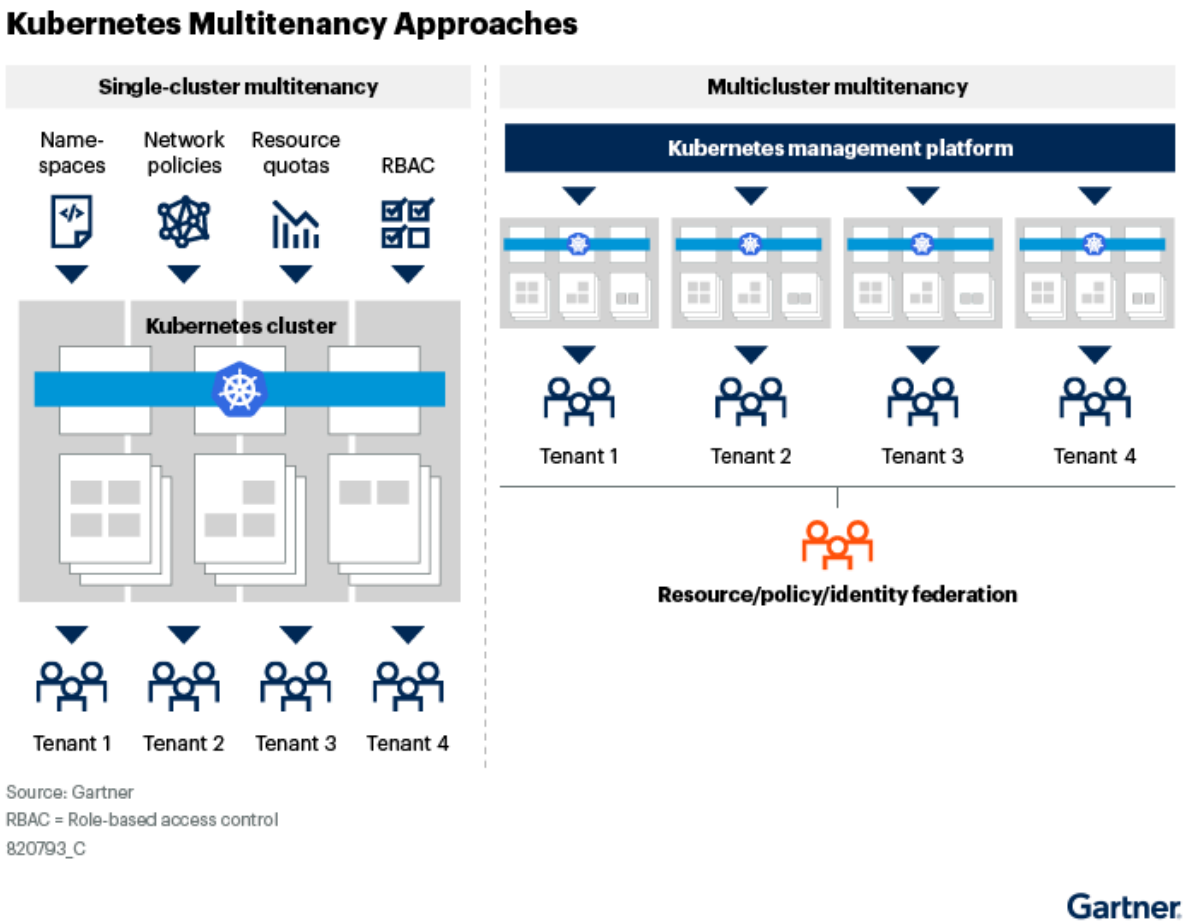
With this use case, separate clusters are deployed in one location for different teams or classes of applications. Kubernetes provides a standard partitioning function called namespace that makes it possible to create a named scope of operation for administrative actions so that cluster resources can be divided between different "tenants" of the cluster. However, namespaces do not provide hard isolation between tenants, and without the most rigorous and time-consuming implementations of safeguards, they serve primarily as an operational convenience for teams who trust each other sharing a cluster.

Furthermore, namespaces do not provide protection against catastrophic failures that affect an entire cluster. [2] Kubernetes is designed to maximize availability of application services within a single cluster, and the reliability of a cluster's control plane can be increased by distributing control plane nodes across multiple availability zones on-premises or in a public cloud. However, it is possible that certain conditions can cause a Kubernetes cluster to become inaccessible or unsecured, or unable to start new applications. In some cases, a seemingly minor fault condition can trigger a chain reaction that causes a cluster to become unresponsive, bringing down all tenants of the cluster. Note that these risks do not just affect multitenancy deployments; single-tenant services may also have to be split, to reduce the impact when a major outage occurs or a service component hits scalability limitations.

To limit the "blast radius" in the event a catastrophic outage makes an entire cluster unresponsive, operators can deploy separate clusters for each tenant. Each tenant is given its own cluster, and a management platform is used to automate the provisioning and deprovisioning of each tenant's cluster. The platform may also federate any resource definitions, policies and identities that have to be shared across clusters (see Figure 5).

This research note is restricted to the personal use of .

Figure 5: Kubernetes Multitenancy Approaches



In the cloud, assigning separate clusters to different teams also simplifies cost showback and chargeback. These requirements are best addressed with a fleet of clusters based on the same distribution, as the common implementation of clusters across the fleet enables optimizations to be shared. Using the same distribution across the fleet also makes it more practical to leverage certain technical enhancements facilitating multitenancy, such as virtual control planes.

DR Use Case

This research note is restricted to the personal use of .

With this use case, multiple clusters are maintained in separate locations so that one cluster can take over the applications of another when a primary location fails or becomes inaccessible. Kubernetes is designed to provide high availability for application services within a single cluster, allowing the services to continue operating after either hardware or software failure. Service operators specify the desired state of a service with a declarative approach that defines how many pod instances should be running. If a pod fails, the pod will be restarted automatically, either on the same node or another worker node with available resources. If a worker node fails, Kubernetes will automatically reschedule the pods running on that node to other nodes.

For resiliency of a cluster's control plane, the nodes hosting the control plane software are usually deployed in quorums of three, five or more to ensure the control plane's operation is not susceptible to any single points of failure or "split brain" conditions. However, the control plane's dependence on the etcd key-value store to maintain its state makes it very sensitive to both network latency and bandwidth between control plane nodes. Stretching the control plane of a single Kubernetes cluster across some distance requires a guaranteed round-trip time (RTT) latency of less than 10ms and the use of three availability zones.

The practical constraints in distance between control plane nodes of a Kubernetes cluster therefore limit its ability to provide protection from disaster-level events. To implement DR for Kubernetes-based applications, it is easier to deploy a separate cluster remotely, and then implement some mechanism to replicate the state of one cluster to another so the backup cluster can resume operations after a disaster event. Two clusters deployed in separate locations are required to support active/passive DR strategies. Active/active DR solutions require clusters in at least three locations, along with a "witness" mechanism that uses a consensus protocol to determine which instances of the cluster are healthy.

To support the DR use case, the fleet management solution must provide the means to configure the replication of application states between clusters, as well as replication of data for stateful applications that manage directly using a PVC. Therefore, the homogeneous cluster fleet management approach will be the best fit for this use case. For further insight on ensuring resilience for containerized applications running on Kubernetes, see Backup and Disaster Recovery for Containers Running on Kubernetes.

Edge Deployment Use Case

This research note is restricted to the personal use of .

Containers and Kubernetes are becoming a popular method for deploying applications at the edge because of the consistency that they provide between development and production environments. This consistency will be particularly valuable where the production environment is remote or otherwise difficult to access, or impractical to be administered locally by nonexperts. The use of containers will increase confidence that applications behave as expected in these environments. In addition, the immutability of containers enables applications to be delivered and updated more easily and safely, using the same CI/CD processes used for creating cloud-native applications in the public cloud.

When Kubernetes is deployed at the edge to orchestrate containers, new tools and operational processes are required to manage fleets of remote Kubernetes clusters. Some edge use cases may require hundreds or thousands of Kubernetes clusters to be managed remotely. Containers were originally designed to work with full connectivity to the life cycle. However, at the edge, containers must be able to operate with a variety of possible network limitations, including:

- Deployments behind a firewall
- Internet connections with very low bandwidth
- Intermittent connectivity to the Internet
- Deployment in physically isolated, aka air-gapped, environments without any Internet connectivity, requiring container images and artifacts to be transported by hand to the edge location

To support the edge deployment use case, the fleet management solution should be able to work with remote Kubernetes clusters under these network conditions. Given the inherent technical challenges with the edge use case, a homogeneous cluster management approach will be the best fit, especially if the vendor of the distribution has a packaged offering for edge deployment. As an alternative, I&O technical professionals can consider a custom approach for the edge use case if they have sufficient technical expertise to work with the open-source tools needed to engineer the solution. For further insight on deploying containers and Kubernetes at the edge, see Deploying Containers and Kubernetes at the Edge.

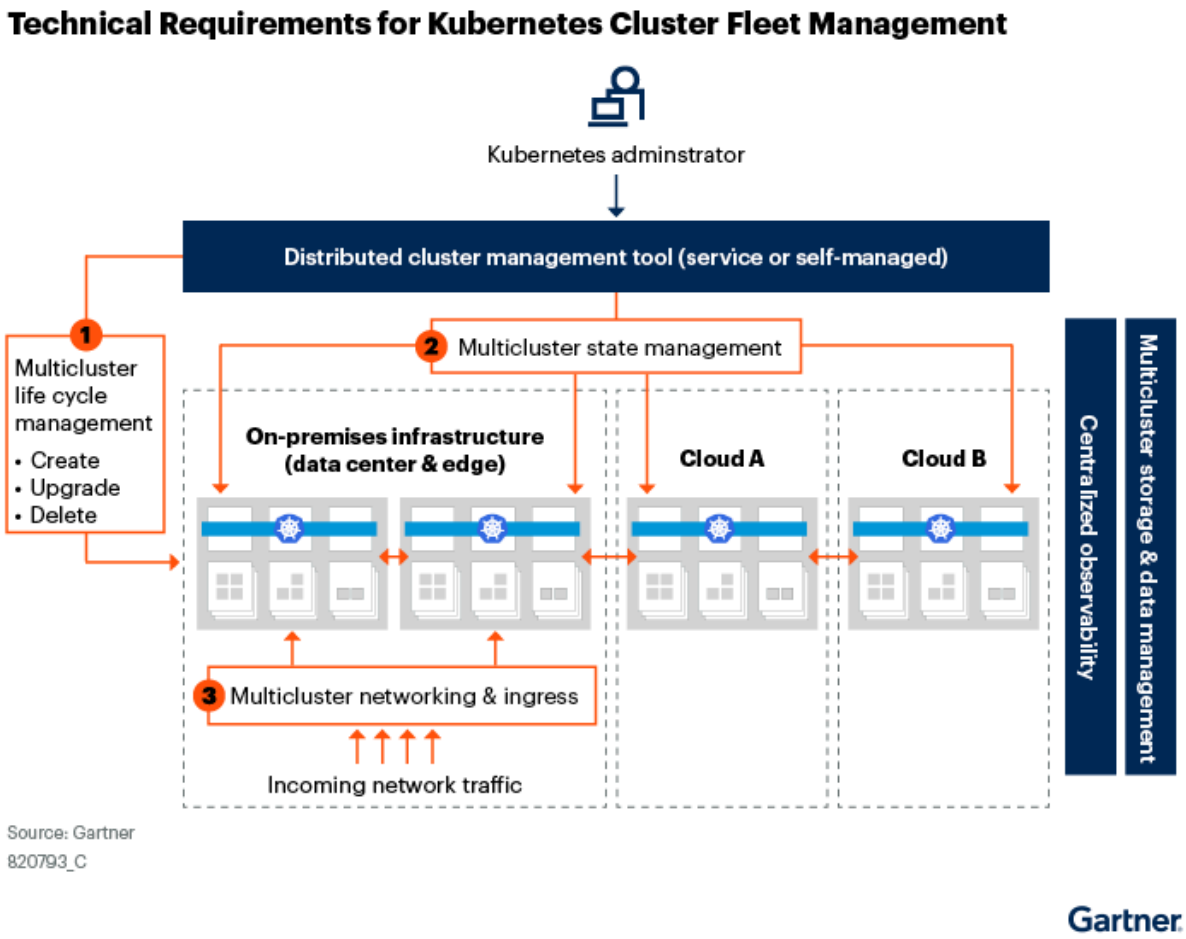
Technical Requirements for Kubernetes Fleet Management

A complete Kubernetes fleet management solution must address several technical requirements (see Figure 6). These requirements include:

This research note is restricted to the personal use of .

- **Multicluster life cycle management,** with the ability to create, upgrade and delete clusters in a fleet on-demand
- **Multicluster state management,** enabling deployments of applications, policies and cluster extensions to be applied consistently across the entire fleet
- **Multicluster networking and ingress,** enabling the distribution of network traffic across multiple clusters
- **Related operations,** including centralized observability, and storage and data management

Figure 6: Technical Requirements for Kubernetes Cluster Fleet Management



Multicluster Life Cycle Management

This research note is restricted to the personal use of .

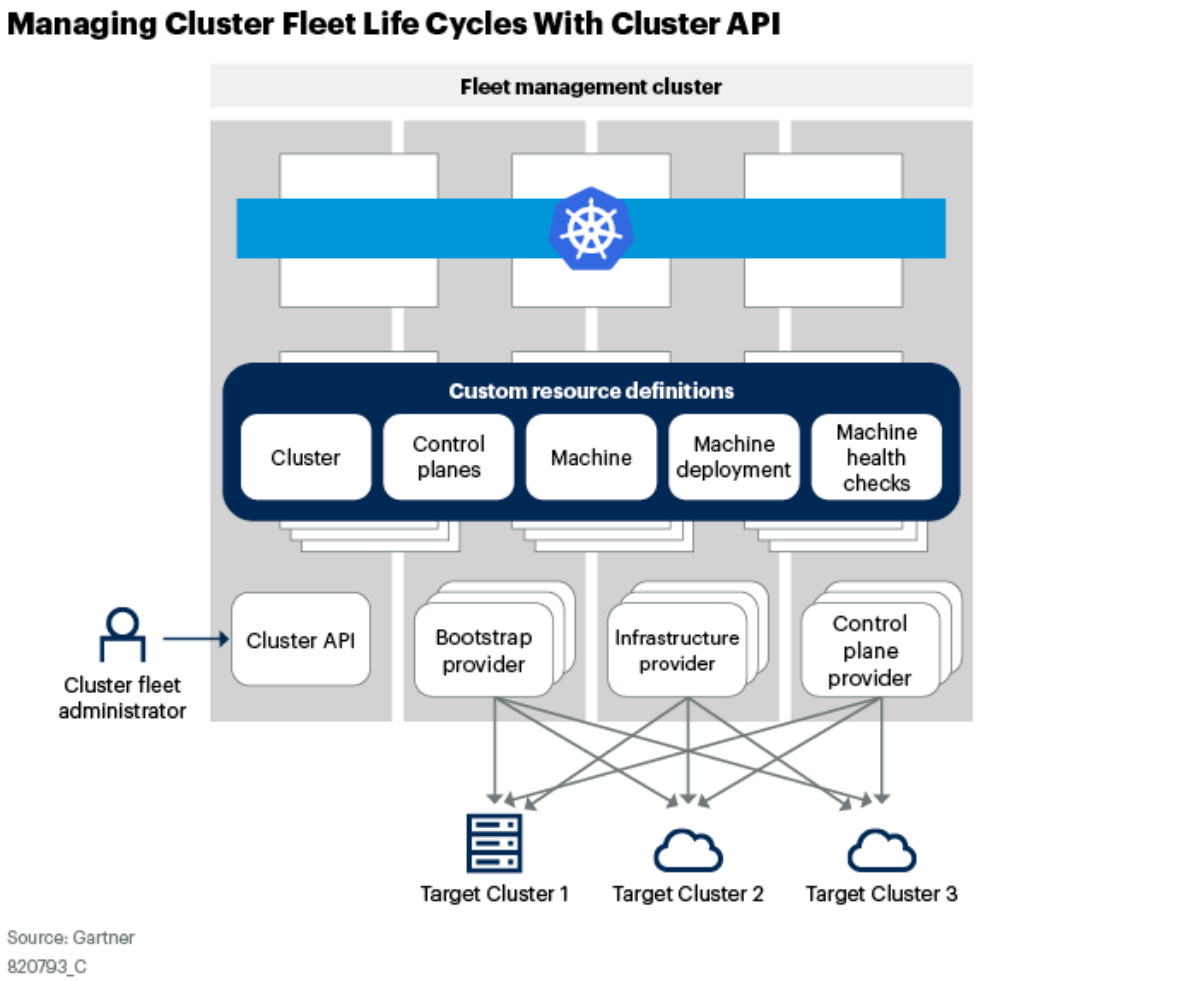
The first requirement for a Kubernetes fleet management solution is to automate the remote provisioning and deprovisioning of Kubernetes cluster software, and upgrade the software on clusters as needed. The Kubernetes software is based on a very active upstream open-source project, and the Kubernetes source code continues to evolve rapidly. One of the main “Day 2” challenges with operating a Kubernetes platform after initial installation is performing consistent upgrades with minimum disruption of production workloads. The upgrade is difficult enough with one cluster, and the challenge is significantly magnified when upgrades have to be applied across a fleet of clusters.

When a fleet of Kubernetes clusters is being upgraded, it should be possible to selectively update the software initially on a subset of the fleet in order to test new releases of Kubernetes software on a canary or blue/green basis. Also, the OS software may be replaced more frequently on worker nodes than control plane nodes in order to apply security updates as quickly as possible. Therefore, it should be possible to remotely upgrade Kubernetes worker node software independently from the control plane node software.

Infrastructure automation tools such as HashiCorp Terraform or Red Hat Ansible Automation Platform can be used for cluster life cycle management, managing Kubernetes software deployments the same way they manage other types of infrastructure software. As a more novel alternative, Cluster API (CAPI) provides a declarative set of Kubernetes-style APIs and patterns for cluster creation, configuration, management and deletion, so that Kubernetes clusters can be provisioned using the same approach that Kubernetes itself uses to provision application services. Using CAPI, fleet administrators can manage the life cycle of Kubernetes clusters and their foundational infrastructure components (e.g., nodes, networks, load balancers and VPCs) through Custom Resource Definitions (CRDs) that operate custom controllers (see Figure 7).

This research note is restricted to the personal use of .

Figure 7: Managing Cluster Fleet Life Cycles With Cluster API



The custom controllers are called providers and are implemented and available for a wide range of on-premises infrastructure or cloud services. These providers, as well as a growing number of provider choices, are now available for different Kubernetes distributions and infrastructure foundations. CAPI allows operators to declaratively manage the clusters' whole life cycle (creation, configuration, management, upgrades and cluster deletion) on any supported infrastructure provider on-premises or in the cloud using Kubernetes-native constructs. In addition to the CLI, practitioners can leverage familiar tools (e.g., kubectl, Helm, Kustomize, Argo CD, etc.) for operating Kubernetes applications to manage clusters' life cycle.

Multicluster State Management

This research note is restricted to the personal use of .

Once fleets of clusters are deployed, it will be necessary to replicate the state of clusters, including application and policy deployments, in order to enable unified governance and policy-based workload management across the cluster fleet. Doing this sequentially for multiple clusters is time-consuming and error-prone, and multicluster state management tooling can alleviate this burden. Centralized cluster state management can also help prevent configuration drift, promote consistency of performance and enable an appropriate DR strategy.

Some of the scenarios for federation include:

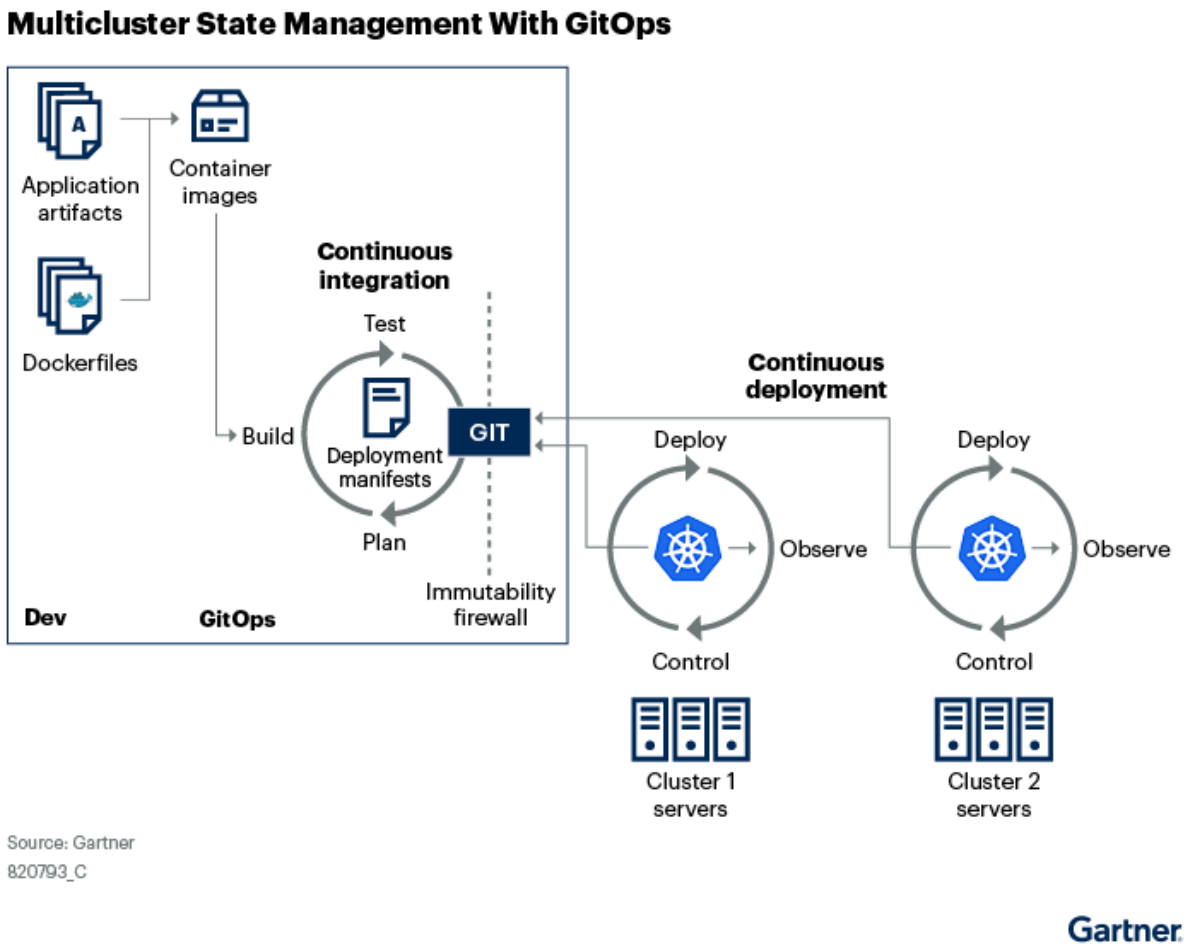
- **Centralized application management:** This provides the ability for administrators to provision, deprovision and update common cluster packages, add-ons and custom-developed software across multiple clusters. This capability needs to work with several other functions, including multicluster ingress controllers, service meshes and service catalogs.
- **Centralized policy management:** This provides the ability to replicate resource policies between clusters. For example, if an application is moved from one cluster to another, it is necessary to make sure the class of service with which the application was deployed on one cluster is also available on the other.
- **Shared services for multitenancy:** When separate clusters are assigned to different tenants, some services and cluster extensions (such as ingress controllers) should be made available to all tenants.
- **State replication for DR purposes:** When backup clusters are set up to recover operations in case a primary cluster fails, the secondary clusters should be configured identically to the primary cluster in order to resume operations quickly.

While administrators may generally prefer a consistent Kubernetes experience across locations, they may also want to define discrete policies and management constructs for cloud, on-premises and edge clusters separately to address the unique parameters of a particular location. For this reason, the federation mechanism in a fleet management solution should allow flexibility in setting the cluster state.

For organizations with a high level of DevOps maturity, multicluster state management can be implemented most effectively with distributed GitOps. With this approach, a centralized Git repository serves as a single source of truth for manifests used to configure the state of all clusters in the fleet (see Figure 8).

This research note is restricted to the personal use of .

Figure 8: Multicluster State Management with Distributed GitOps



On each cluster, a GitOps agent watches over the changes in the Git repository, constantly monitoring the declared state of deployment manifest files against the actual state of the target clusters. To deploy a new version of an application or policy, the updated manifest only has to be checked into the Git repository and the updated deployment is then automatically pulled to each cluster. This approach also prevents drift; if any discrepancies arise due to out-of-band changes applied to a cluster, the GitOps agent will automatically reverse them by restoring the application or policy state that is currently checked into the Git repository.

This research note is restricted to the personal use of .

Note that with heterogeneous cluster fleets, the multicluster state management solution must be able to account for differences between the Kubernetes distributions. For example, in hybrid scenarios, clients may want to deploy the same containerized applications in clusters running on-premises and in the cloud, whereby they are running different distributions in each domain. In this case, the workflow for building and delivering the application must recognize which distribution is being targeted in each cluster, and factor in differences in objects that might be unique to a distribution. Such objects include ingress controllers, storage resources, identity mechanisms, etc., in the manifest applied to each cluster.

Multicluster Networking and Ingress

In some fleet scenarios, it will be necessary to plan how services running on different clusters will interact. Containerized applications usually depend on a cluster's container network interface (CNI) to communicate within the cluster. Multicluster networking will require the use of CNIs that have the ability to route traffic between clusters, such as Cilium, Istio or Submariner.

When organizations adopt a multicluster architecture with Kubernetes, different tiers of a multitiered application may be distributed across multiple clusters. In practice, however, a number of questions can arise about the feasibility of distributing application services across multiple clusters, including:

- What placement restrictions apply to a particular cluster based on compliance, risk domains or data proximity for stateful containerized applications?
- What are load balancing and/or traffic routing requirements for the application?
- What are the service discovery requirements related to deploying the application (i.e., how will its services be publicized to consumers)?
- What are the requirements for container registry locality (i.e., the ability to efficiently access container images needed for starting the application)?

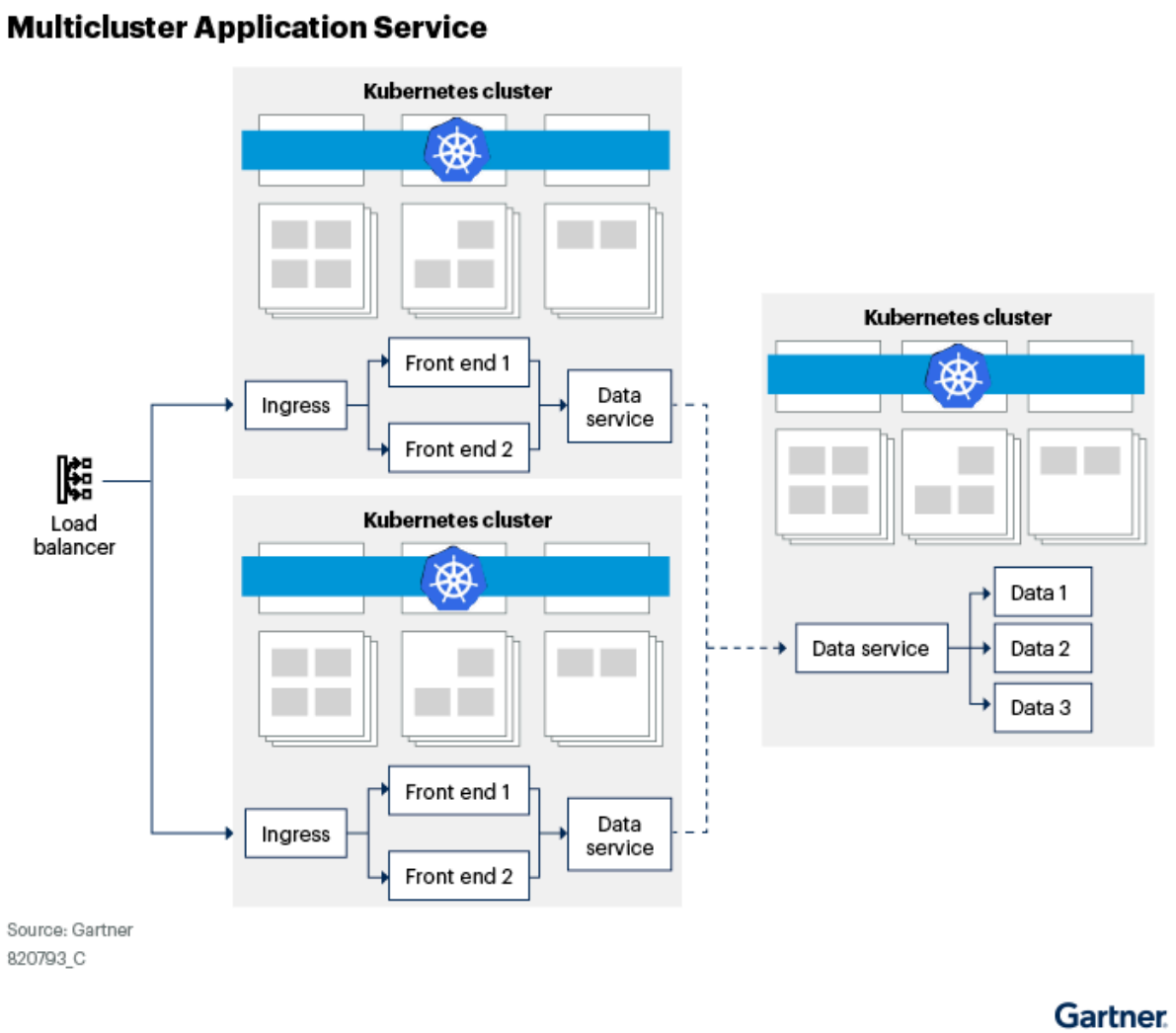
The most straightforward approach for distributing applications across multiple clusters is to use

separate clusters for different tiers of a multitiered application. This approach allows the full complement of Kubernetes service management capabilities to be applied to each tier independently, while maintaining isolation between each tier for maximum fault containment.

This research note is restricted to the personal use of .

For example, as shown in Figure 9, a two-tier application might load-balance its front-end services across two separate clusters that are accessed through a multicluster-enabled load balancer, which distributes traffic across the two clusters. The front-end clusters then share access to a third cluster hosting the application's back-end service, (Note that this approach, or something similar, would be required for an active-active DR configuration.)

Figure 9: Example of Multicluster Application Service



Several open-source projects provide multicluster networking, including:

- Skupper, which is a Layer 7 service interconnect that enables secure communication between different Kubernetes clusters

This research note is restricted to the personal use of .

- KubeSlice, which creates a flat, secure virtual network overlay for streamlined data distribution and communication between distributed workloads running on different Kubernetes clusters

Another approach for deploying multicluster application services works the application layer by leveraging a service mesh such as Istio, Linkerd or HashiCorp Consul. These service meshes can manage and monitor service-to-service communications running on different clusters, usually in support of a microservices architecture.

Related Operations

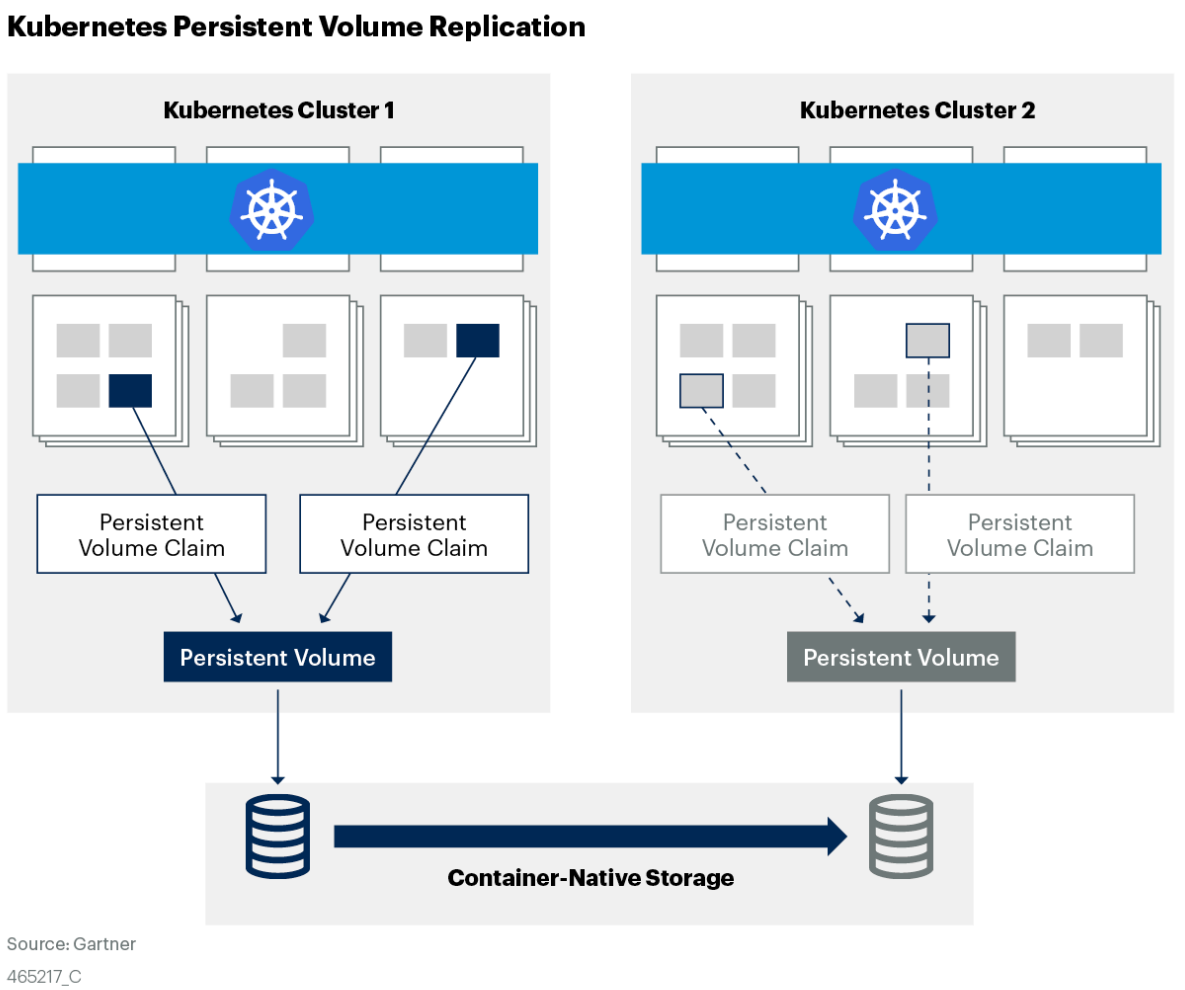
Multicluster Storage and Data Management

When stateful applications migrate from one cluster to another, the relationship with underlying storage resources requires careful management using robust processes that can coordinate their binding. When clusters are located in different regions, the migration of data becomes even more challenging. The Kubernetes storage abstractions enable persistent volumes (PVs) to be rebound to PVCs, as pods running stateful applications migrate from one node to another within a single cluster.

However, these bindings do not extend to migrating an application from one cluster to another. It is straightforward to replicate the storage resources underlying PVs from one cluster to another, along with the Kubernetes resources needed to restart an application service on the other cluster. However, the challenge lies in reestablishing the correct binding between the migrated application and its data, since the Kubernetes architecture does not define any federated method for specifying PVCs and PVs (see Figure 10).

This research note is restricted to the personal use of .

Figure 10: Kubernetes Persistent Volume Replication



Therefore, replicating PVs between clusters requires a distributed abstraction layer for container-native data management. Open-source projects such as Velero provide a streamlined solution for backing up multiple types of Kubernetes resources, including PVs, so that the state of an entire cluster can be migrated from one region or data center to another. In addition, a new generation of container-native storage (CNS) solutions are designed to manage data exclusively for containerized applications, and some of these are designed to support data management across multiple clusters.

Centralized Observability

A Kubernetes fleet management solution should provide a dashboard that can be used to observe the behavior and operational efficiency of all the clusters in the fleet. The management solution should also provide centralized monitoring and alerting of issues that may arise with the functioning of a cluster or the applications running on it.

This research note is restricted to the personal use of .

In practice, centralized monitoring is one of the more straightforward management functions to implement across multiple clusters. Prometheus has become established as the de facto standard for monitoring and alerting in the container ecosystem. Its pull-based API and multidimensional data format provides a center of gravity for component integrations. Prometheus federation allows a centralized Prometheus server to scrape selected time series data from other distributed Prometheus servers, enabling its operation to scale across multiple regions and large numbers of Kubernetes worker nodes. Many container security products are designed for multicluster operation, and they offer the ability to monitor and secure the behavior of multiple clusters from one pane of glass. For further insight on observability with Kubernetes, see How to Monitor Containers and Kubernetes Workloads.

Kubernetes Fleet Management Approaches

There is no standard way to deploy and operate multiple Kubernetes clusters. Thus, any cluster fleet management solution will require either purchasing a third-party product or service, or putting together a custom solution that has to be developed and maintained internally. A key question that arises is whether the solution will be based on multiple clusters running the same Kubernetes distribution or on different distributions.

Table 1 shows representative commercial offerings and open-source tools that are most optimized for each of these approaches. Note that some offerings also support capabilities for another approach, such as homogeneous fleet management solutions having some support for heterogeneous Kubernetes distributions, even as they are primarily optimized for native distribution.

This research note is restricted to the personal use of .

Table 1: Examples of Solutions for Kubernetes Fleet Management Approaches

Homogeneous Kubernetes Cluster Fleet Management Offerings	Heterogeneous Kubernetes Cluster Fleet Management Offerings	Open-Source Tools Supporting Custom Kubernetes Fleet Management Approaches
▪ Broadcom (VMware Tanzu Kubernetes Grid) ▪ Giant Swarm ▪ Google GKE Enterprise ▪ Kubermatic ▪ Microsoft Azure Kubernetes Fleet Manager ▪ Mirantis ▪ Platform9 ▪ SUSE (Rancher) ▪ Red Hat Advanced Cluster Management for Kubernetes	▪ Broadcom (VMware Tanzu Mission Control) ▪ Diamanti Spektra ▪ Elotl (Luna) ▪ Rafay Systems ▪ Spectro Cloud ▪ SUSE (Rancher)	▪ Admiralty ▪ Gardener ▪ Karmada (Kubernetes Armada) ▪ KubeStellar ▪ Open Cluster Management

Source: Gartner (November 2024)

Homogeneous Kubernetes Cluster Fleet Management Solutions

As managing multiple Kubernetes clusters becomes a growing priority for enterprise organizations, Kubernetes fleet management capabilities have become a feature in Kubernetes distributions targeting that set of customers. With this approach, vendors provide a solution optimized for their own Kubernetes distribution, and their fleet management capabilities may be integrated with other capabilities they provide related to containers and cloud-native architecture, such as developer tools or cloud services.

As a disadvantage, the homogeneous approach may be opinionated on other layers of infrastructure, such as the host operating system, virtualization, on-premises infrastructure or cloud infrastructure. This opinionation may limit customers' choices in terms of selecting these layers with their own solution.

This research note is restricted to the personal use of .

Some examples of vendors offering homogeneous Kubernetes cluster fleet management solutions include:

- **Broadcom,** which supports Kubernetes cluster fleets at several levels with VMware Tanzu. The Tanzu Kubernetes Grid (TKG) architecture is fundamentally based on the concept of managing multiple Kubernetes clusters using the same declarative APIs that Kubernetes normally uses to orchestrate containers. With this approach, VMware Tanzu effectively uses Kubernetes itself to manage Kubernetes clusters.
- **Giant Swarm,** which provides a managed Kubernetes service that enables customers to run multiple fully managed Kubernetes clusters on top of a private infrastructure on-premises or public IaaS cloud infrastructure (Amazon Web Services [AWS] and Microsoft Azure).
- **Google,** which supports federating policies across multiple GKE instances in GKE Enterprise. This capability can be used to manage fleets of clusters deployed in GCP, on-premises in a customer's data center, or on other cloud services such as AWS. GKE Enterprise Fleets are logical groups of Kubernetes clusters that can be managed together using the Connect gateway, which enables administrators to connect to and run commands against fleet member clusters
- **Kubermatic,** which automates the operation of multiple Kubernetes clusters in the cloud or on-premises, with a single management UI to set up, manage and operate the clusters.
- **Microsoft,** which can group AKS clusters into fleets using Azure Kubernetes Fleet Manager to automate upgrades and set up distributed cluster operations such as configuration propagation. It can be used with Azure Kubernetes Service Edge Essentials, an on-premises implementation of AKS with a small footprint and simple installation experience.
- **Mirantis,** which provides a cluster management tool called k0smotron which can run and manage multiple Kubernetes cluster control planes as pods within a single Kubernetes management cluster. The tool integrates with CAPI to support tasks such as provisioning, scaling and upgrading clusters.
- **Platform9,** which provides a managed Kubernetes service that can be deployed on-premises and on public clouds. The SaaS-based centralized service allows organizations to manage multiple Kubernetes clusters in a multitenant environment. Clusters can be in different locations that are geographically distributed.

This research note is restricted to the personal use of .

- **SUSE,** with its Rancher multicluster management platform. Rancher provides a central console for deploying multiple clusters in different locations, and performs life cycle management operations like upgrades and rollback.
- **Red Hat,** with its Red Hat Advanced Cluster Management (RHACM), which can provision and manage multiple Red Hat OpenShift clusters from a single console. While RHACM is designed to manage Red Hat OpenShift clusters, it can also discover and onboard existing clusters based on other Kubernetes distributions so that their state can be managed in the same domain as Red Hat OpenShift.

Note that some of these products offer interoperability with heterogeneous Kubernetes distributions, with the ability to "attach" clusters running other distributions and perform state management operations on those clusters for application or policy configuration. However, they support the full range of life cycle management capabilities, as well as advanced configuration with GitOps or Operators, only with their own distribution.

Heterogeneous Kubernetes Cluster Fleet Management Solutions

With growing interest in hybrid and multicluster container deployments, a number of management products have emerged that are designed specifically to manage clusters running heterogeneous Kubernetes distributions. The advantage of these solutions is that they provide independence from specific cloud platforms and, in some cases, a consistent operational experience across on-premises and cloud infrastructure. However, some of these products address only a subset of the technical requirements for fleet management described previously.

Some examples of vendors offering heterogeneous Kubernetes cluster fleet management offerings include:

- **Broadcom,** which supports heterogeneous cluster fleets with VMware Tanzu Mission Control (TMC), an API-driven centralized management platform for Kubernetes infrastructure, with the ability to manage cluster life cycles and state configuration in cluster fleets. TMC can be used to manage the life cycle of Amazon AWS Elastic Kubernetes Service (EKS) and AKS, and it can attach to clusters based on other Kubernetes distributions for centralized operations.
- **Diamanti,** which provides a central interface to manage cluster fleets with its Spektra platform. Spektra uses an architecture based on Domain clusters to manage Tenant clusters. Domain and Tenant clusters can be deployed using either EKS or AKS, and clusters running Red Hat OpenShift can be attached to Tenant clusters.

This research note is restricted to the personal use of .

- **Elotl,** which offers a multicluster control plane called Nova that provides policy-driven orchestration across multiple Kubernetes clusters. Nova supports several cloud-based Kubernetes services, including EKS, GKE, AKS and Oracle OKE, and on-premises distributions (Red Hat OpenShift and VMware Tanzu).
- **Rafay Systems,** which automates Kubernetes management and governance across public cloud, data centers and edge. Its platform enables centralized administration and life cycle management of Kubernetes clusters running either its own Kubernetes distribution or other distributions deployed in the cloud or on-premises. Key product features include automated provisioning and life cycle management of managed clusters, as well as cluster blueprinting and drift detection, Kubernetes software add-on management, and cluster multitenancy management.
- **Spectro Cloud,** which provides a software platform for the management of large numbers of distributed Kubernetes clusters in the cloud, on-premises data centers, or distributed across thousands of individual nodes or clusters at the edge. Spectro Cloud's Palette architecture uses cluster profiles to specify a complete stack as a declarative model, from the OS up to Kubernetes clusters and containerized applications.
- **SUSE,** which can manage the life cycle of multiple Kubernetes distributions in addition to its own distributions using Rancher (Rancher Kubernetes Engine [RKE] for data centers and K3s for edge deployments). Rancher's other supported distributions include EKS, GKE, AKS, Alibaba ACK, Tencent Kubernetes Engine (TKE), and Huawei Cloud Container Engine (CCE).

Tools for Custom Kubernetes Fleet Management Solutions

For organizations that have sufficient engineering resources, it may be possible to create a custom solution for managing cluster fleets and deploying containerized applications on them. The advantage of this approach is that the organization can target specific requirements for their particular use cases, and the solution may not need to address all the technical requirements described previously. However, with this approach, the organization takes on the responsibility of integrating all the required components for the solutions and then maintaining that integration, which will require deep development expertise and familiarity with the open-source ecosystem.

If this approach is selected, some possible open-source tools to consider for a customized cluster fleet management solution include:

This research note is restricted to the personal use of .

- **Admiralty** is a system of Kubernetes controllers that intelligently schedules workloads across multiple Kubernetes clusters by connecting the control planes of the clusters.
- **Gardener** is an extension API server and a set of custom controllers to enable automated management and operation of Kubernetes clusters as a service. Gardener introduces new API objects in an existing Kubernetes cluster (called a "garden cluster") in order to use them for the management of end-user Kubernetes clusters (called "shoot clusters").
- **Karmada (Kubernetes Armada)** is a Kubernetes management system that uses a Kubernetes-native API definition to create federated resource templates that can be used to define multicluster scheduling and application propagation requirements.
- **KubeStellar** is a Cloud Native Computing Foundation (CNCF) Sandbox project that simplifies the deployment and configuration of applications across multiple Kubernetes clusters. KubeStellar allows developers to define a binding policy between distributed clusters and Kubernetes objects. It then uses standard cluster management tools to deploy and configure each cluster based on these binding policies, making multicluster operations as straightforward as managing a single cluster.
- **Open Cluster Management (OCM)** is a CNCF Sandbox project to build a modular, extensible platform for Kubernetes multicluster orchestration. It is based on Hub Clusters, which are lightweight Kubernetes clusters hosting a few fundamental controllers and services, and Klusterlet (aka "spoke") clusters that are managed by the Hub Clusters.

Guidance

The following guidelines will enable success with deploying and operating distributed Kubernetes

clusters given the current state of container and Kubernetes technology:

This research note is restricted to the personal use of .

- **Scope cluster resources appropriately.** Do not try to implement a single scalable cluster that can satisfy the requirements of all possible application services. Research by Google has shown that excessive consumption of resources by one class of objects will start to constrain resources that can be used by others. [3] Therefore, it is important to understand the resource requirements of the application mix that will be deployed on a particular cluster. Scope the size and infrastructure of the cluster based on the expected consumption of objects such as pods, namespaces, services and ingress controllers. Identify the constraints that overconsumption of one object type will place on the others.

- **Identify application exposure to cluster failure.** When planning cluster topology, determine the risk of applications being exposed to the catastrophic failure of a shared cluster. Scope shared clusters for hosting applications based on the minimum cause of risk that is acceptable for the applications deployed on the cluster. Structure the necessary operational processes for DR spanning multiple clusters, either in the same or different regions.

- **Target cluster instance immutability.** Define abstract classes of clusters that can be used to launch specific cluster instances on-demand. Implement processes for deploying containerized applications that treat clusters like “cattle,” meaning they are not dependent on any specific attributes of a particular cluster.

- **Automate cluster life cycle management.** Set up procedures for creating clusters using tools that can be scripted, and establish CI/CD pipelines for automating their provisioning, deprovisioning and upgrades. As an alternative, use a commercial product offering Kubernetes management federation capabilities to help manage the life cycle of distributed clusters.

- **Leverage CAPI to provision clusters at scale.** Target Kubernetes infrastructure based on its support for CAPI and select fleet management solutions that implement CAPI to facilitate Kubernetes life cycle management at scale. CAPI provides a consistent method to manage the entire life cycle of clusters using Kubernetes-native constructs on any supported infrastructure provider.

- **Federate security policies with core and edge topology.** Using cluster state management tooling, define tiers of cluster classes with different security classifications, ranging from trusted to semitrusted and untrusted clusters tiers. Limit trust and control access to “core” clusters with resources such as root certificates. Use methods for distributing security policies to propagate the policies outward toward edge clusters with gradually decreasing levels of trust.

This research note is restricted to the personal use of .

- **Mirror application topology with cluster topology.** Use separate clusters for different tiers of a multitiered application to maintain isolation between each tier for maximum fault containment. Take advantage of automated deployment and cluster instance immutability to implement autoscaling of cluster instances, in which clusters are automatically allocated and deallocated in response to changing workloads and availability conditions.

- **Plan for distributed data management.** When containerizing stateful applications, develop a data management strategy that accounts for the persistence services that are available on each cluster, and can support application portability and DR between clusters.

- **Align image and artifact management with cluster topology.** Hybrid deployments may necessitate a distributed approach where discrete applications are deployed on-premises with controls defined uniquely from cloud deployments. The cached distribution of container registries and other artifacts generally needs to be deployed in proximity to deployments for improved pull performance. Implement distributed container registries as well that can be used to cache container images for more efficient access by the distributed clusters. If artifacts like YAML files or Helm charts are needed for deployment and/or operation of distributed clusters, implement local instances of artifact management tools, such as revision control systems, that can cache these artifacts for more efficient access by distributed clusters. Note that in some verticals, such as telecom, media and manufacturing, this approach doesn't always work because of the distributed responsibility across multiple vendors.

- **Ensure sufficient resources when deploying Kubernetes clusters at the edge.** Some use cases will not support the deployment of complete Kubernetes clusters at the edge, due to limited infrastructure resources. In such cases, it is preferable to keep Kubernetes running in data centers or the cloud, while orchestrating containers on modes deployed at the edge. Because the Kubernetes architecture assumes continuous connectivity between the cluster control plane and worker nodes, this approach will require specialized configurations of Kubernetes clusters.

This research note is restricted to the personal use of .

- **Evaluate the trade-offs between reliability and simplicity.** Note that fleet-based solutions for multitenancy introduce more administration complexity compared with shared clusters. The fleet-based solution also assumes that tenant teams have expertise in using the Kubernetes API for orchestrating containers. Gartner has found this approach to be suitable in engineering-driven organizations where knowledge of and experience with using Kubernetes is widespread. However, teams that are not as mature in the adoption of cloud-native infrastructure may prefer a serverless approach, in which details about Kubernetes are abstracted away using a shared environment. When planning multitenancy strategies, it is necessary to decide which approach to adopt, or possibly offer a choice of both.

Evidence

[1] vCluster and kcp.

[2] For an example of a global Kubernetes failure resulting from a chain reaction of distributed management subsystems becoming unresponsive, see On Infrastructure at Scale: A Cascading Failure of Distributed Systems, Medium.

[3] Kubernetes Scalability: A Multi-Dimensional Analysis, KubeCon 2018, YouTube.

Recommended by the Author

Some documents may not be available as part of your current Gartner subscription.

How to Implement Cluster Fleet Management for Complex Kubernetes Deployments

Solution Path for Cloud-Native Infrastructure With Kubernetes

Guidance Framework for Deploying Containers and Kubernetes

Is Using GitOps-Style Automation With Kubernetes Right for Me?

A Layered Approach to Implement Container Platforms On-Premises

DevOps Workflows to Deploy Container Applications With Kubernetes

This research note is restricted to the personal use of .

© 2026 Gartner, Inc. and/or its affiliates. All rights reserved. Gartner is a registered trademark of Gartner, Inc. and its affiliates. This publication may not be reproduced or distributed in any form without Gartner's prior written permission. It consists of the opinions of Gartner's Business and Technology Insights Organization, which should not be construed as statements of fact. While the information contained in this publication has been obtained from sources believed to be reliable, Gartner disclaims all warranties as to the accuracy, completeness or adequacy of such information. Although Gartner insights may address legal and financial issues, Gartner does not provide legal or investment advice and its insights should not be construed or used as such. Your access and use of this publication are governed by Gartner's Usage Policy. Gartner prides itself on its reputation for independence and objectivity. Its insights is produced independently by its Business and Technology Insights Organization without input or influence from any third party. For further information, see " Guiding Principles on Independence and Objectivity." Gartner insights may not be used as input into or for the training or development of generative artificial intelligence, machine learning, algorithms, software, or related technologies.

This research note is restricted to the personal use of .

Table 1: Examples of Solutions for Kubernetes Fleet Management Approaches

Homogeneous Kubernetes Cluster Fleet Management Offerings	Heterogeneous Kubernetes Cluster Fleet Management Offerings	Open-Source Tools Supporting Custom Kubernetes Fleet Management Approaches
■ Broadcom (VMware Tanzu Kubernetes Grid)	■ Broadcom (VMware Tanzu Mission Control)	■ Admiralty
■ Giant Swarm	■ Diamanti Spektra	■ Gardener
■ Google GKE Enterprise	■ Elotl (Luna)	■ Karmada (Kubernetes Armada)
■ Kubermatic	■ Rafay Systems	■ KubeStellar
■ Microsoft Azure Kubernetes Fleet Manager	■ Spectro Cloud	■ Open Cluster Management
■ Mirantis	■ SUSE (Rancher)	
■ Platform9		
■ SUSE (Rancher)		
■ Red Hat Advanced Cluster Management for Kubernetes		

Source: Gartner (November 2024)

This research note is restricted to the personal use of .